

Reporte consolidado del estudio del CRB de la cámara de esquina (NLOS)

Qué se intentó en las cuatro ramas, qué resultó,
y en qué estado queda el repositorio

Repositorio crb

29 de agosto de 2026

Resumen

Este documento consolida en un solo lugar todo el trabajo realizado sobre el límite de Cramér–Rao (CRB) de la localización de un objeto oculto con una cámara de esquina NLOS, repartido hasta ahora en cuatro ramas de trabajo con sus respectivos *pull requests*. El objetivo es que, al retirar esas ramas, no se pierda ni el razonamiento ni los números: se recogen aquí la cronología, las decisiones de modelado, los resultados numéricos de cada intento, los hallazgos de la revisión matemática y de código, y el estado final del repositorio.

El estado final adopta tres decisiones. Primero, el simulador de transientes se reemplaza por una implementación vectorizada en PyTorch que procesa bloques de triángulos contra todos los píxeles del SPAD a la vez. Segundo, la grilla de referencia («*baseline*») pasa de 64×64 a 32×32 píxeles. Tercero, el CRB de producción es el de diferencias finitas sobre ese simulador, con vector de parámetros $\psi = (\rho, \varphi)$ y matriz de información de Fisher 2×2 : el simulador nuevo no expone la altura del objeto como entrada, de modo que σ_h deja de estar definida. Con esa configuración, σ_ρ recorre 1,35 mm–9,0 cm y σ_φ recorre $0,75^\circ$ – $11,7^\circ$ sobre la rejilla de poses $\rho \in \{0,5, 1,0, 1,5\}$ m, $\varphi \in \{30, 60, 90, 120, 150\}^\circ$.

Índice

1. Contexto y objetivo	2
1.1. El problema físico	2
1.2. La pregunta que se quería responder	2
1.3. Qué se hizo, en una frase	2
2. Qué es el CRB y cuáles son las dos rutas	2
2.1. Fisher de Poisson y el CRB	2
2.2. El obstáculo: el <i>forward</i> no es diferenciable	3
2.3. Ruta FD: diferenciar numéricamente el simulador	3
2.4. Ruta analítica: suavizar y derivar exacto	3
3. Cronología: qué se intentó en cada rama	4
3.1. PR#1 — Regeneración de las regiones CRB por la ruta FD	5
3.2. PR#2 — <i>Forward</i> analítico diferenciable (base cualitativa)	5
3.3. PR#3 — Amplitud física y rejilla del simulador (a)+(b)	6
3.4. PR#4 — Suma sobre la malla real densificada (a)+(b)+(c)	7
4. Hallazgos de la revisión	9
4.1. Lo que se verificó correcto	10
4.2. Claims corregidos	10
4.3. Hallazgos menores	11

5. Estado final y decisiones	12
5.1. Cómo se conectan el CRB y el simulador nuevo	12
5.2. Resultados del CRB por FD con el <i>baseline</i> 32×32	13
5.3. Qué quedó en el repositorio y qué se retiró	14
6. Catálogo de figuras	15
6.1. Documentos originales y dónde quedan sus contenidos	16
7. Cómo reproducir el estado final	16

1. Contexto y objetivo

1.1. El problema físico

Una cámara de esquina NLOS ilumina un punto del suelo con un láser pulsado y observa, con un sensor SPAD que resuelve el tiempo de vuelo, la mancha de luz que regresa tras rebotar. Si hay un objeto fuera de la línea de vista (detrás de la esquina), parte de la luz hace un camino de dos rebotes láser $\rightarrow$ objeto $\rightarrow$ suelo $\rightarrow$ SPAD, y su tiempo de llegada y su distribución angular contienen información sobre dónde está el objeto.

El simulador del repositorio implementa exactamente ese modelo directo («*forward*»). Para cada triángulo t de la malla de la escena y cada píxel p del SPAD acumula una contribución

$$I_{t,p} = I_{\text{láser}} \cdot \text{área}_t \cdot \frac{\cos \theta_1 \cos \theta_2 \cos \theta_3 \cos \theta_4}{4\pi^2 d_1^2 d_2^2} \cdot \underbrace{\mathbf{1}[x_{\text{int}} > 0]}_{\text{no oclusión}}, \quad (1)$$

en el *bin* temporal $j = \lceil (d_1 + d_2)/(c \Delta t) \rceil$, donde d_1 es la distancia láser–triángulo, d_2 la distancia triángulo–píxel, los cuatro cosenos son productos escalares recortados a cero, y x_{int} es la abscisa del corte con el eje de la recta triángulo–píxel, que codifica si la arista de la esquina oculta ese camino.

1.2. La pregunta que se quería responder

La pregunta es de precisión alcanzable, no de reconstrucción: *dada esta geometría y este sensor, ¿cuál es la mínima varianza que puede tener cualquier estimador no sesgado de la posición del objeto?* Eso es el CRB. La respuesta se dibuja como una región de incertidumbre 3σ en el plano de parámetros (ρ, φ) (coordenadas polares del objeto sobre el suelo), una elipse —una «burbuja»— por cada pose evaluada.

1.3. Qué se hizo, en una frase

Se construyeron y compararon dos rutas para calcular ese CRB (la de diferencias finitas sobre el simulador y la de un *forward* analítico diferenciable), se llevó la segunda hasta reproducir a la primera dentro de un 10 %, se sometió todo a una revisión matemática y de código independiente, y finalmente se reemplazó el simulador por una versión vectorizada en GPU y se fijó la grilla de referencia en 32×32 .

2. Qué es el CRB y cuáles son las dos rutas

2.1. Fisher de Poisson y el CRB

El SPAD cuenta fotones: el número de cuentas en el *bin* i es una variable de Poisson de media $g_i(\psi)$, donde $\psi = (\rho, \varphi)$ (o (ρ, φ, h) en las versiones que estimaban también la altura) y

$$g_i(\psi) = y_i(\psi) + b, \quad (2)$$

con y_i la señal esperada del modelo directo y $b > 0$ una tasa de fondo constante (en todo el estudio $b = 0,1$ cuentas/bin). Para observaciones de Poisson independientes, la matriz de información de Fisher es

$$\mathbf{I}(\psi) = \sum_i \frac{1}{g_i(\psi)} \frac{\partial g_i}{\partial \psi} \frac{\partial g_i}{\partial \psi}^\top = \mathbf{J}^\top \text{diag}(1/g) \mathbf{J}, \quad \mathbf{J}_{im} = \frac{\partial g_i}{\partial \psi_m}, \quad (3)$$

y el CRB es su inversa, $\text{Cov}(\hat{\psi}) \succeq \mathbf{I}^{-1}$. De $\mathbf{I}^{-1}$ se leen $\sigma_\rho = \sqrt{[\mathbf{I}^{-1}]_{11}}$, $\sigma_\varphi = \sqrt{[\mathbf{I}^{-1}]_{22}}$, el error tangencial $\rho \sigma_\varphi$, y la elipse 3σ se obtiene por descomposición espectral de $\mathbf{I}^{-1}$.

El papel de $b > 0$ es doble: es física (hay fondo real) y es matemático, porque mantiene g_i acotado lejos de cero y por tanto el peso $1/g_i$ finito.

2.2. El obstáculo: el *forward* no es diferenciable

Todo el problema del CRB se reduce a obtener el jacobiano $\mathbf{J} = \partial g / \partial \psi$. Pero el *forward* de la ecuación (1) contiene tres operaciones que destruyen la diferenciable:

- (I) el **binning temporal** $\lceil \cdot \rceil$, constante a trozos: mover el objeto un poco no cambia nada hasta que el tiempo de vuelo cruza un borde de *bin*, y entonces una masa entera salta de un *bin* al siguiente;
- (II) la **máscara de oclusión** $\mathbf{1}[x_{\text{int}} > 0]$, un escalón de Heaviside;
- (III) los **recortes** $\text{máx}(0, \cdot)$ de los cuatro cosenos, con un codo no diferenciable en el origen.

De ahí las dos rutas.

2.3. Ruta FD: diferenciar numéricamente el simulador

Se toma el simulador tal cual (con sus escalones) y se aproxima cada columna del jacobiano por una diferencia central,

$$\mathbf{J}_{:,m} \approx \frac{g(\psi + \delta_m \mathbf{e}_m) - g(\psi - \delta_m \mathbf{e}_m)}{2\delta_m}, \quad (4)$$

con pasos $\delta_\rho = 0,01$ m y $\delta_\varphi = 0,5^\circ$. Es la ruta de producción: vive en `crb_polar_functions.py` y usa el modelo de malla completo, sin aproximaciones de modelado. Su debilidad es numérica: sobre un *forward* escalonado la diferencia finita mezcla mesetas (derivada cero) con saltos de *bin* (derivada $\propto 1/\delta$), y el resultado depende del paso δ . Lo que la rescata en la práctica es la densificación de la malla: con miles de triángulos las distancias por triángulo se reparten y el jacobiano agregado se auto-promedia.

2.4. Ruta analítica: suavizar y derivar exacto

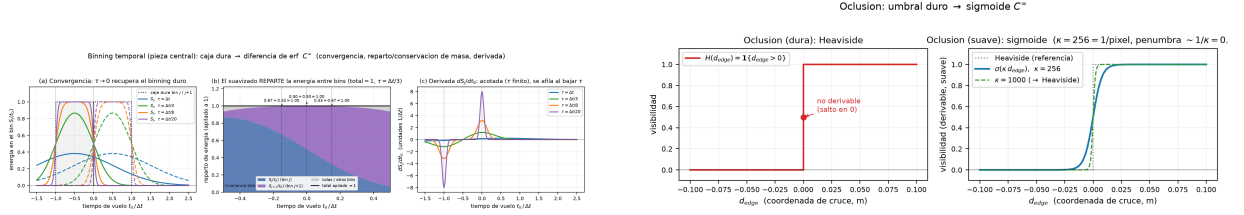
Se reemplazan las tres operaciones no diferenciables por versiones C^∞ con un parámetro de anchura, se construye el *forward* suave en `sympy`, y se deriva de forma *exacta* y simbólica:

Operación dura	Reemplazo suave	Anchura
$\lceil \cdot \rceil$ (masa en un <i>bin</i>)	diferencia de erf	τ
$\mathbf{1}[x_{\text{int}} > 0]$ (oclusión)	sigmoide $\sigma(\kappa x_{\text{int}})$	$1/\kappa$
$\text{máx}(0, x)$ (recortes)	<i>softplus</i> $\log(1 + e^{\beta x})/\beta$	$1/\beta$

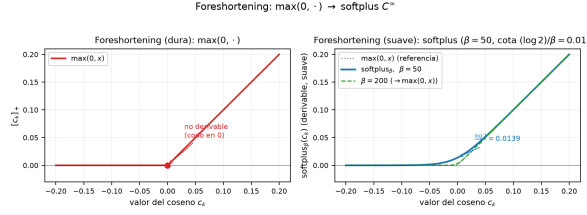
La justificación del primer reemplazo es la que da nombre a la construcción: en lugar de asignar toda la masa del triángulo al *bin* que contiene su tiempo de vuelo t_0 , se reparte con un núcleo gaussiano de anchura τ y se integra sobre el *bin* $[j\Delta t, (j+1)\Delta t)$, lo que da

$$S_{ij}(t_0) = \frac{1}{2} \left[\text{erf}\left(\frac{(j+1)\Delta t - t_0}{\sqrt{2}\tau}\right) - \text{erf}\left(\frac{j\Delta t - t_0}{\sqrt{2}\tau}\right) \right], \quad (5)$$

que es real-analítica en t_0 , suma exactamente 1 sobre todos los *bins* (conserva la masa) y converge al *binning* duro cuando $\tau \rightarrow 0$. Las figuras 1 muestran los tres reemplazos.



(a) *Binning* temporal: $[\cdot] \rightarrow \text{erf}$ (ecuación (5)). (b) Oclusión: escalón $\rightarrow$ sigmoide de penumbra.



(c) Recorte de los cosenos: $\max(0, \cdot) \rightarrow \text{softplus}$.

Figura 1: Los tres reemplazos que convierten el *forward* del simulador en una función C^∞ derivable en forma cerrada. Figuras heredadas de la rama de suma sobre malla, con la configuración final $\kappa = 256$, $\tau = \Delta t / \sqrt{12}$.

Sobre esa base, el documento `analytic_forward_crb.tex` desarrollaba una justificación matemática completa: los bloques «suaves» del simulador (distancias, tiempos de vuelo, cosenos) ya son C^∞ fuera de degeneraciones (lema de la norma euclídea); el diagnóstico de la no diferenciabilidad se hace con el **teorema de la función implícita** para identificar la superficie de salto y con el **teorema de Rademacher** para situar el conjunto de no diferenciabilidad; las herramientas para el paso al límite son la **convergencia dominada**, la **derivación bajo la integral** (Leibniz) y las **aproximaciones de la identidad** (*mollifiers*); y el cierre es que el CRB es C^∞ donde $\mathbf{I} \succ 0$. Un resultado negativo importante de ese desarrollo es la **inconsistencia de la diferencia finita en un salto**: en el límite duro la derivada diverge y la Fisher «dura» no existe en sentido clásico, lo cual es precisamente la razón de suavizar en lugar de intentar derivar el modelo duro.

3. Cronología: qué se intentó en cada rama

Las cuatro ramas son incrementales: cada una añade un ingrediente a la anterior y se mide contra la misma referencia FD.

PR	Rama	Aporte	Resultado en una línea
#1	regenerate-crb-ellipse-plots	regenera los PNG de la ruta FD como burbujas en (ρ, φ)	solo figuras; no toca código
#2	analytic-differentiable-forward	<i>forward</i> analítico suave, base cualitativa	comparable solo en <i>forma</i>
#3	analytic-forward-physical-grid	(a) amplitud física + (b) rejilla del simulador	$\sigma_\rho^{\text{an}} / \sigma_\rho^{\text{FD}} \in [0,50, 0,85]$; sesgo de cuadratura puntual $\sim 1,5\times$
#4	analytic-forward-mesh-sum	(a)+(b)+(c) suma sobre la malla real densificada	razón $\sigma_\rho \in [1,08, 1,10]$; razón $\sigma_\varphi \in [1,06, 1,12]$

3.1. PR#1 — Regeneración de las regiones CRB por la ruta FD

Qué se hizo. Se regeneraron las figuras del CRB calculado por diferencias finitas para que se dibujaran como *elipses en el plano de parámetros* (ρ, φ) —burbujas centradas en cada pose— en lugar del empuje hacia adelante («*pushforward*») al plano físico xy , que producía formas de gota difíciles de leer. En el código esto corresponde a llamar `plot_crb_regions_polar(..., use_physical_region=False)`, con `rlim=2.0`, etiquetas radiales a dos decimales y fuentes reducidas.

El cálculo se hacía con el *script* `regenerate_crb_standard_regions.py`, que cargaba las celdas 1, 3, 5, 7, 8 del *notebook* `simulation_polar_clean.ipynb` para obtener la función `simulation` y sus parámetros, y luego llamaba a `compute_crb_grid_polar` dos veces: una con $\psi = (\rho, \varphi)$ (Fisher 2×2) y otra con $\psi = (\rho, \varphi, h)$ (Fisher 3×3).

Qué se aprendió. El diff de esta rama frente a `main` era exactamente cuatro PNG: los que el *script* escribe. La revisión confirmó que la representación en el plano de parámetros es la correcta para leer σ_ρ y σ_φ directamente de los semiejes. El código del CRB por FD ya vivía en `main` (`crb_polar_functions.py`); PR#1 no lo modificaba.

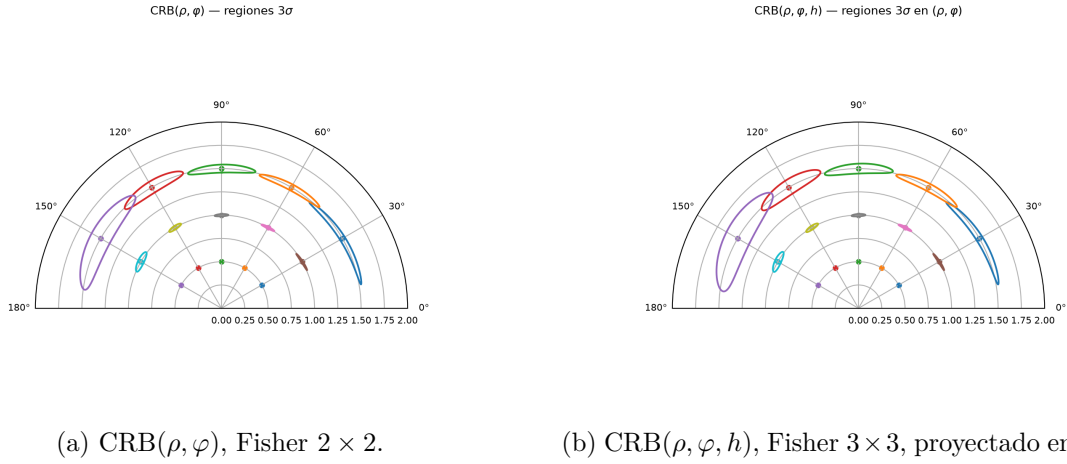


Figura 2: PR#1: regiones 3σ de la ruta FD sobre el simulador antiguo (64×64 , `subpixel_dim=4`). Estimar además la altura h ensancha las burbujas, porque la información se reparte entre tres parámetros.

3.2. PR#2 — *Forward* analítico diferenciable (base cualitativa)

Qué se hizo. Se construyó por primera vez el *forward* suave completo en `sympy`, con los tres reemplazos de la sección 2, y su jacobiano exacto. El módulo es `crb_analytic_forward.py` y el documento que lo justifica es `docs/analytic_forward_crb.tex`, con el desarrollo paso a paso (construcción, derivada, justificación matemática con teoremas, Fisher de Poisson, validación y resultados).

Esta rama es deliberadamente una **base cualitativa**. Su configuración no pretende ser fiel al simulador:

- la faceta se modela como un **punto** colocado en $z = h$ (no en el centroide de la malla, que está en $\approx h/2$);
- la amplitud se pliega en una **ganancia libre** $G = 10^5$ constante, de modo que $\partial y / \partial h$ representa «subir una faceta de tamaño fijo», no el área $\propto h$ del simulador;
- usa su **propia rejilla** 8×8 con FOV 0,5 y centro $-0,25$, en lugar de la 64×64 / FOV 0,25 / centro $-0,125$ del simulador;

- las anchuras de suavizado son genéricas: $\kappa = 60$, $\tau = \Delta t$.

Qué se aprendió. La validación intrínseca funcionó a la primera y con margen: el jacobiano simbólico coincide con una diferencia central del *mismo forward* suave con error máximo $5,0 \cdot 10^{-10}$. Es decir, la derivada exacta está bien implementada. Pero las magnitudes absolutas del CRB **no son comparables** con la ruta FD, porque los modelos son distintos y G es arbitraria: la comparación honesta es de *forma*, tras normalizar la escala. Con esa normalización, la relación de aspecto media resultó 4,96 frente a 8,22 del FD, y la inclinación de las elipses fue consistente.

La conclusión operativa de PR#2 fue: la derivada exacta es claramente mejor condicionada que la FD (que sufre el ruido de discretización de los escalones), pero para que el CRB analítico signifique algo hay que ponerlo en las unidades y la geometría del simulador. Eso motivó PR#3 y PR#4.

Tabla 1: PR#2: CRB analítico (A) frente a FD (F) sobre la rejilla estándar. Las columnas σ analíticas *no* son comparables en magnitud con las FD (ganancia G libre, geometría y rejilla propias); lo comparable es **asp** (eje mayor/eje menor) y la inclinación.

ρ [m]	φ [°]	Analítico (base cualitativa)			FD (referencia)		
		σ_ρ [m]	σ_φ [°]	asp	σ_ρ [m]	σ_φ [°]	asp
0.50	30	0.001306	0.4259	6.85	0.0006859	0.5091	19.21
0.50	60	0.001184	0.2891	4.50	0.0006779	0.4401	14.03
0.50	90	0.001390	0.2838	3.64	0.0007805	0.3846	9.61
0.50	120	0.002027	0.2448	2.13	0.001116	0.4116	7.08
0.50	150	0.003547	0.3478	1.72	0.001847	0.5484	5.70
1.00	30	0.001382	0.5349	9.84	0.003406	2.012	13.06
1.00	60	0.001201	0.3740	6.17	0.003448	1.478	8.23
1.00	90	0.001400	0.3807	5.01	0.004197	1.416	6.19
1.00	120	0.001992	0.3231	2.88	0.006299	1.460	4.20
1.00	150	0.003395	0.4680	2.43	0.01086	2.214	3.70
1.50	30	0.002255	0.8964	10.86	0.01207	6.763	12.10
1.50	60	0.001918	0.6378	6.76	0.01215	4.687	7.23
1.50	90	0.002228	0.6684	5.59	0.01486	4.664	5.66
1.50	120	0.003150	0.5646	3.19	0.02185	4.717	3.84
1.50	150	0.005356	0.8426	2.77	0.03747	7.432	3.51
Aspecto medio		4,96			8,22		

3.3. PR#3 — Amplitud física y rejilla del simulador (a)+(b)

Qué se hizo. Dos correcciones sobre PR#2:

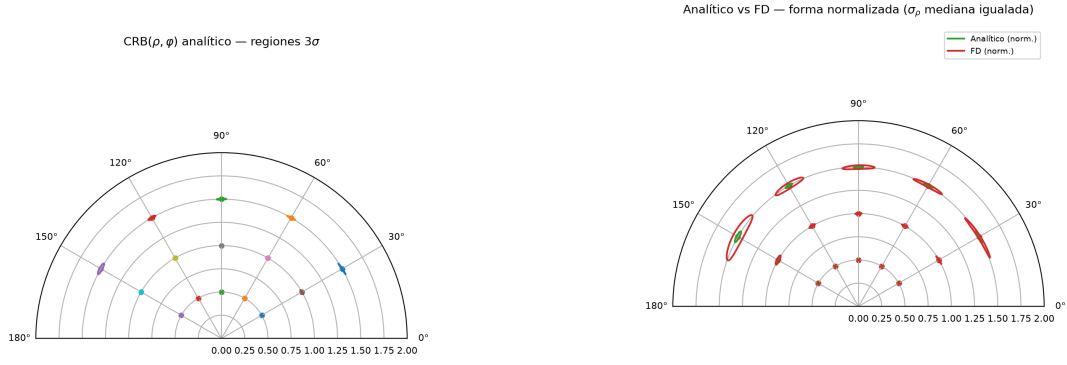
(a) Amplitud física.

Se elimina la ganancia libre $G = 10^5$ y se pone la amplitud real del simulador: $I_{\text{láser}} = 1000$ y área efectiva $\text{área}_{\text{eff}} = w \cdot h$ con $w = 0,5$ m fijo y h la altura estimada, de modo que la magnitud escala con h igual que en la malla. No queda ningún parámetro de ganancia libre.

(b) Rejilla del simulador.

Se adopta la rejilla real: 64×64 píxeles, FOV 0,25, centro $-0,125$, y el tamaño de *bin* verdadero $\Delta t = 3,9 \cdot 10^{-10}$ s. El centroide de la faceta se coloca en $z = h/2$, como en la malla.

Qué se aprendió: el hallazgo central de esta rama. La corrección funcionó para lo que buscaba — σ_ρ y σ_φ analíticos quedaron en las *mismas unidades* que los FD, y del mismo orden— pero reveló un problema de modelado más profundo:



(a) Burbujas del CRB analítico base, escala propia. (b) Superposición normalizada en forma: analítico (verde) vs FD (rojo).

Figura 3: PR#2: al no ser comparables las magnitudes, la superposición se hace escalando ambas familias de elipses al mismo σ_ρ mediano. El escalado es isótropo alrededor del centro, así que preserva aspecto e inclinación.

Hallazgo 1 (Sesgo sistemático de cuadratura puntual). Evaluar la radiometría en **un solo punto** (el centroide, con área $w \cdot h$) es una regla del punto medio para una integral de superficie, y **sobreestima la intensidad en un factor** $\approx 1,5$ frente a la suma sobre la malla. Medido en $\psi = (1 \text{ m}, 75^\circ, 1 \text{ m})$: la energía total puntual da 2617 frente a 1749 de la suma sobre 512 triángulos, una razón de 1,50, con un error L_1 por píxel de $\approx 50\%$. La causa dominante es la **oclusión**: el centroide está visible para casi todos los píxeles mientras que parte de la faceta real de $0,5 \times 1 \text{ m}$ está ocluida. A eso se añade la no linealidad de $1/d_2^2$ y de los cosenos sobre una faceta extendida a distancias de $\sim 1 \text{ m}$.

Como consecuencia, $\sigma_\varphi^{\text{an}}/\sigma_\varphi^{\text{FD}}$ baja hasta $\approx 0,57$ y el rango global de $\sigma_\rho^{\text{an}}/\sigma_\rho^{\text{FD}}$ queda en $[0,50, 0,85]$.

Hallazgo 2 (Endurecer el suavizado *no* corrige el sesgo). Una afirmación inicial de esta rama era que al llevar $\tau \rightarrow 0$, $1/\kappa, 1/\beta \rightarrow 0$ el CRB analítico «converge hacia el FD». Es **falso**. Al endurecer, esta variante converge a su *propio forward* duro de *faceta puntual* (verificado: el error L_1 por píxel baja de 15,9% a 0,34% respecto a ese duro puntual), **no** al *forward* de malla del simulador. El error de cuadratura es sistemático y sobrevive a cualquier endurecimiento. La única corrección es sumar sobre la malla, que es exactamente lo que hace PR#4.

Un efecto secundario, más pequeño, es que las anchuras de suavizado seguían siendo genéricas: $\tau = \Delta t$ y $\kappa = 60$ dan una penumbra de $1/\kappa = 1,67 \text{ cm}$, que son $\approx 4,3$ píxeles del simulador ($\text{pitch } 0,25/64 = 0,39 \text{ cm}$). Es decir, más gruesas que la discretización que pretenden imitar, y parte del desajuste de forma (aspecto medio 9,36 frente a 8,22 del FD) viene de ahí.

3.4. PR#4 — Suma sobre la malla real densificada (a)+(b)+(c)

Qué se hizo. Se añadió el ingrediente que faltaba:

(c) Suma sobre la malla real.

En lugar de un punto, el *forward* analítico suma sobre los triángulos reales de `facet.obj` **densificado a 512 triángulos**, con el *placement* (colocación) hecho de forma **simbólica**: escala anisótropa $\rightarrow$ *pitch* $\rightarrow$ *roll* $\rightarrow$ *yaw* $\varphi + 3\pi/2 \rightarrow$ elevación por $z_{\min} \rightarrow$ traslación. El área, la normal y el centroide de cada triángulo quedan en forma cerrada, y el ensamblado se factoriza como $g = \sum_t A^{(t)} S^{(t)} + b$, con derivada $dA \cdot S + A \cdot (dS/dt_0) \cdot dt_0$.

Tabla 2: PR#3 (a)+(b): ahora las columnas σ sí están en las mismas unidades que las FD. La razón analítico/FD queda sistemáticamente por debajo de 1 ([0,50,0,85] en σ_ρ), que es la huella del sesgo de cuadratura puntual del hallazgo 1.

ρ [m]	φ [°]	Analítico (a)+(b)			FD (referencia)			razón σ_ρ
		σ_ρ [m]	σ_φ [°]	asp	σ_ρ [m]	σ_φ [°]	asp	
0.50	30	0.0005795	0.3799	15.26	0.0006859	0.5091	19.21	0.845
0.50	60	0.0005580	0.2838	10.13	0.0006779	0.4401	14.03	0.823
0.50	90	0.0006579	0.2717	7.75	0.0007805	0.3846	9.61	0.843
0.50	120	0.0009266	0.2364	4.65	0.001116	0.4116	7.08	0.830
0.50	150	0.001541	0.3056	3.58	0.001847	0.5484	5.70	0.834
1.00	30	0.002008	1.407	17.83	0.003406	2.012	13.06	0.589
1.00	60	0.001912	1.078	11.57	0.003448	1.478	8.23	0.555
1.00	90	0.002288	1.115	9.25	0.004197	1.416	6.19	0.545
1.00	120	0.003268	0.9953	5.52	0.006299	1.460	4.20	0.519
1.00	150	0.005424	1.379	4.56	0.01086	2.214	3.70	0.499
1.50	30	0.008991	6.318	18.20	0.01207	6.763	12.10	0.745
1.50	60	0.008545	4.875	11.74	0.01215	4.687	7.23	0.703
1.50	90	0.01034	5.253	9.64	0.01486	4.664	5.66	0.696
1.50	120	0.01500	4.821	5.79	0.02185	4.717	3.84	0.687
1.50	150	0.02504	6.950	4.94	0.03747	7.432	3.51	0.668
Aspecto medio		9,36			8,22			

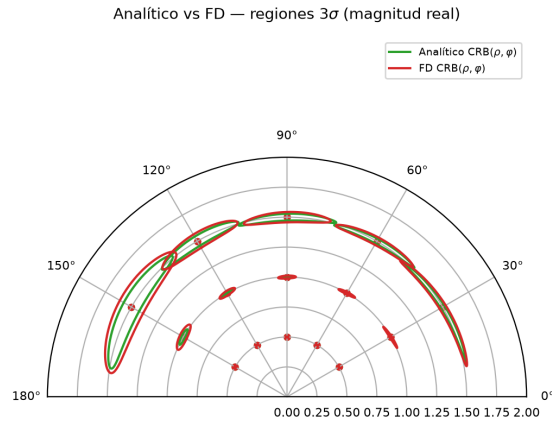


Figura 4: PR#3: superposición a magnitud real (sin normalizar) del CRB analítico (a)+(b) sobre el FD. Las escalas ya coinciden, pero las elipses analíticas son sistemáticamente más pequeñas: el modelo puntual «ve» más señal de la que hay y por tanto promete más precisión de la real.

Anchuras atadas a la discretización.

$\kappa = 1/\text{pixel_pitch} = 256$ (penumbra de exactamente un píxel), lo que fija σ_φ ; y $\tau = \Delta t/\sqrt{12}$ (la misma varianza que el *top-hat* que implementa el $[\cdot]$), lo que fija σ_ρ .

Qué se aprendió. Esta variante es la más fiel de las tres. La verificación independiente reprodujo la celda 8 del *notebook* con **numpy**: el *placement* simbólico replica el *pipeline* de **trimesh** a precisión de máquina y la energía total coincide al 0,03 %. Y el resultado del CRB es el buscado:

$$\frac{\sigma_\rho^{\text{an}}}{\sigma_\rho^{\text{FD}}} \in [1,08, 1,10], \quad \frac{\sigma_\varphi^{\text{an}}}{\sigma_\varphi^{\text{FD}}} \in [1,06, 1,12], \quad (6)$$

en *toda* la rejilla y en *ambos* ejes, con aspecto medio 8,27 frente a 8,22 del FD. Es decir, las dos rutas —numéricamente muy distintas— coinciden dentro de un 10 %, lo que es una validación cruzada fuerte de las dos.

Los dos ingredientes que lo consiguen son separables y ambos necesarios: densificar la faceta (con pocos triángulos la suma es una cuadratura de superficie demasiado gruesa) y atar las anchuras de suavizado a la discretización del simulador.

Se documentaron además dos limitaciones numéricas reales, ambas *fuera* de la rejilla publicada $\varphi \in [30^\circ, 150^\circ]$: un polo de oclusión que producía valores no finitos en el jacobiano para $\varphi \lesssim 20^\circ$ (resuelto enmascarando a cero las contribuciones no finitas por (triángulo, píxel), el análogo directo de la máscara **valid** del simulador), y el desbordamiento del *softplus* $\log(1 + e^{\beta x})/\beta$ cuando $\beta x \gtrsim 709$, que limita el endurecimiento a $\beta \approx 700$.

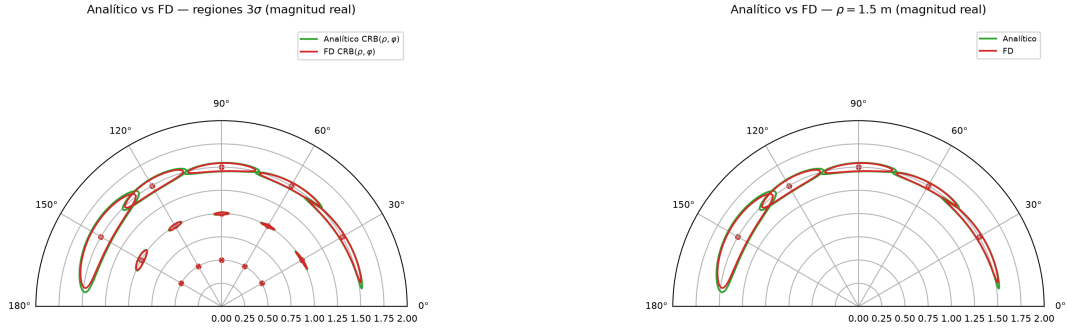
Esta rama contiene también **docs/guia_intuitiva.tex**, una guía en lenguaje llano que explica qué es el CRB y por qué se dibuja como elipse, describe las dos rutas, mapea qué añade cada rama, y cataloga los PNG.

Tabla 3: PR#4 (a)+(b)+(c): la variante más fiel. Ahora ambas razones están por encima de 1 y muy cerca de él: el modelo suave es ligeramente *conservador* respecto al FD, en lugar de optimista como en PR#3.

ρ [m]	φ [°]	Analítico (a)+(b)+(c)			FD (referencia)			razones	
		σ_ρ [m]	σ_φ [°]	asp	σ_ρ [m]	σ_φ [°]	asp	σ_ρ	σ_φ
0.50	30	0.0007502	0.5510	19.07	0.0006859	0.5091	19.21	1.094	1.082
0.50	60	0.0007458	0.4741	13.83	0.0006779	0.4401	14.03	1.100	1.077
0.50	90	0.0008592	0.4125	9.43	0.0007805	0.3846	9.61	1.101	1.073
0.50	120	0.001223	0.4394	6.95	0.001116	0.4116	7.08	1.096	1.068
0.50	150	0.002039	0.5878	5.59	0.001847	0.5484	5.70	1.104	1.072
1.00	30	0.003699	2.229	13.58	0.003406	2.012	13.06	1.086	1.108
1.00	60	0.003722	1.599	8.32	0.003448	1.478	8.23	1.079	1.082
1.00	90	0.004541	1.539	6.26	0.004197	1.416	6.19	1.082	1.087
1.00	120	0.006790	1.564	4.20	0.006299	1.460	4.20	1.078	1.071
1.00	150	0.01173	2.348	3.65	0.01086	2.214	3.70	1.080	1.061
1.50	30	0.01315	7.566	12.59	0.01207	6.763	12.10	1.089	1.119
1.50	60	0.01317	5.179	7.41	0.01215	4.687	7.23	1.084	1.105
1.50	90	0.01618	5.188	5.82	0.01486	4.664	5.66	1.089	1.112
1.50	120	0.02396	5.167	3.85	0.02185	4.717	3.84	1.097	1.095
1.50	150	0.04085	8.015	3.49	0.03747	7.432	3.51	1.090	1.078
Aspecto medio		8,27			8,22				

4. Hallazgos de la revisión

Todo el trabajo anterior se sometió a dos revisiones independientes: una matemática (corrección de los *forwards* analíticos y sus derivadas) y una de código, figuras y documentación (ruta



(a) Rejilla completa.

(b) Detalle en $\rho = 1,5$ m, el caso más difícil.

Figura 5: PR#4: superposición a magnitud real del CRB analítico (a)+(b)+(c) sobre el FD. Las elipses caen una sobre otra sin ninguna normalización, en ambos ejes.

FD). Ambas trabajaron en modo lectura y ejecutaron las validaciones para comprobar que los números citados se reproducen.

4.1. Lo que se verificó correcto

- **Cero errores críticos.** Ninguna de las dos revisiones encontró errores matemáticos o de implementación que invaliden los resultados publicados. No hay errores de signo, de dirección de normales ni de constantes en ninguna rama.
- **Las derivadas simbólicas son correctas** en las tres ramas analíticas: jacobiano `sympy` frente a diferencia central del *mismo forward* suave da error máximo $5,0 \cdot 10^{-10}$ (PR#2), $9,7 \cdot 10^{-10}$ (PR#3) y $5,5 \cdot 10^{-9}$ (PR#4), todos muy por debajo del umbral de 10^{-6} .
- **Los números publicados se reproducen exactamente:** tablas de validación, CRB de referencia, σ de la rejilla y aspecto medio, en las tres ramas.
- **La ruta FD está correctamente implementada:** la diferencia central y la extrapolación de Richardson $(4D(\delta/2) - D(\delta))/3$ son correctas; $\mathbf{I} = \mathbf{J}^\top \text{diag}(1/g)\mathbf{J}$ con piso ε es correcta (y con fondo 0,1 el piso nunca actúa); la elipse 3σ por descomposición espectral es la frontera correcta de $\{\delta^\top \Sigma^{-1} \delta \leq k^2\}$; y la matriz $A = \begin{pmatrix} \cos \varphi & -\rho \sin \varphi \\ \sin \varphi & \rho \cos \varphi \end{pmatrix}$ es el jacobiano de $(x, y) = (\rho \cos \varphi, \rho \sin \varphi)$, bien aplicada como $\Sigma_{xy} = A \Sigma A^\top$.
- **El *placement* simbólico de PR#4 replica exactamente** la celda 8 del *notebook* (matrices de rotación verificadas contra `trimesh.rotation_matrix`), con energía total coincidente al 0,03 %.

4.2. Claims corregidos

Hallazgo 3 (La convergencia $128 \rightarrow 512$ triángulos no era $< 0,5\%$). La documentación afirmaba que «los cambios de 128 a 512 y a 2048 triángulos son $< 0,5\%$ ». Medido en $(\rho = 1, \varphi = 90^\circ)$ con la configuración final: 128 triángulos dan $\sigma_\rho = 0,004248$ y 512 dan 0,004541, es decir $+6,9\%$; el paso $512 \rightarrow 2048$ sí es $+0,37\%$. En σ_φ : $+0,7\%$ y $+0,12\%$. La distinción correcta, que es la que quedó documentada, es que la **energía** converge a ≈ 128 triángulos ($\approx 0,1\%$, medido $0,137\%$, hasta 2048), pero las σ **del CRB** —que dependen de derivadas— necesitan ≈ 512 . La configuración publicada (512) sí está razonablemente convergida, así que los resultados se sostienen; era la justificación escrita la que estaba mal.

Hallazgo 4 (Una *caption* autogenerada contradecía su propia tabla). La función que escribía la tabla LaTeX de PR#4 tenía la *caption* con el rango « σ_ρ anal./FD $\approx 0,9\text{--}1,05$ » **escrito a mano**, mientras que las razones de la tabla adjunta son $1,08\text{--}1,10$ (todas > 1), y el propio abstract decía « $\approx 10\%$ ». Quedó corregido calculando el rango mínimo-máximo a partir de los datos, con lo que ahora imprime $1,08\text{--}1,10$; las filas de la tabla se verificaron byte-idénticas antes y después.

Hallazgo 5 («Converge hacia el FD» era incorrecto en PR#3 y PR#4). Ya recogido como hallazgo 2 para PR#3. En PR#4 la frase «en el límite duro las Fisher coinciden» choca con la propia teoría del documento (la proposición de inconsistencia de la FD: en el límite la derivada diverge y la Fisher dura no existe clásicamente). Como heurística —«ambas rutas ven la misma física»— vale; como enunciado de convergencia, el mismo documento lo refuta.

Hallazgo 6 (El paso FD $\delta_\rho = 0,01$ m frente a la cuantización del *bin*). Un *bin* temporal vale $c \Delta t = 299792458 \times 3,9 \cdot 10^{-10} \approx 0,1169$ m de camino (ida y vuelta), es decir $\approx 0,058$ m en ρ . Con $\delta_\rho = 0,01$ m, la diferencia central abarca $0,04$ m de camino, un 34% **de un bin**: para un triángulo aislado, la mayoría de las evaluaciones $\pm\delta$ caen en el mismo *bin* (meseta, derivada temporal cero) y las que cruzan aportan saltos de masa $\propto 1/(2\delta_\rho)$. Lo que rescata a la ruta FD es la **densificación** de la malla, que reparte las distancias por triángulo y auto-promedia el jacobiano agregado. El estimador es razonable pero ruidoso y dependiente de δ —lo cual es coherente con que el modelo suave de PR#4, con $\tau = \Delta t/\sqrt{12}$, coincida con el FD dentro de un 10% . Corolario práctico: `finite_difference_jacobian_polar_richardson` es matemáticamente inapropiado sobre un *forward* escalonado (la extrapolación de Richardson supone una expansión del error en potencias de δ , que cerca de un salto no existe); no se usa en ningún *script* —siempre **central**—, pero conviene saberlo. Los otros pasos, $\delta_\varphi = 0,5^\circ$ y $\delta_h = 0,01$ m, sí son razonables.

Hallazgo 7 (Fragilidad de las cachés). Dos problemas, ambos corregidos. Primero, la caché analítica `crb_grid_results_analytic.pkl` se reutilizaba a ciegas si contenía las claves esperadas, y las *tres* ramas analíticas usaban el *mismo* nombre de archivo para *tres* modelos distintos: correr un *script* en un *checkout* con la caché de otra variante habría producido figuras y tablas obsoletas en silencio. Se resolvió estampando en el *pickle* una huella de configuración (`config_key`) y recomputando con un aviso explícito si no coincide. Segundo, la caché FD se resolvía por ruta absoluta `/workspace/plots/crb_grid_results.pkl`, y en PR#4 además en tiempo de importación, lo que solo funciona en esa máquina con ese *layout*.

4.3. Hallazgos menores

- El *docstring* de PR#2 afirmaba «re-expresa la MISMA intensidad por píxel» que el simulador, lo cual no es cierto (faceta puntual en $z = h$, ganancia constante, rejilla propia). Se reescribió declarando explícitamente que es la variante base cualitativa, con una sección de salvedades honestas.
- La afirmación « $\kappa = 60$ da una penumbra de $1,7$ cm, comparable al tamaño de píxel» era falsa en ambos sentidos: el píxel de la rejilla propia de PR#2 mide $6,25$ cm y el del simulador $0,39$ cm. Corregida a «valor genérico intermedio, no atado a ninguno».
- Imprecisiones numéricas en la guía intuitiva: «PR#3: $\sigma_\rho \approx 0,5\text{--}0,7 \times$ FD» cuando el rango real es $0,50\text{--}0,85$; y «razón $1,00\text{--}1,12$ » para PR#4 cuando el rango real es $1,08\text{--}1,10$ en σ_ρ y $1,06\text{--}1,12$ en σ_φ . Corregidas.
- `_pick_zmin_ref` usaba *pitch*, *roll* y escala codificados a mano en lugar de leerlos de la configuración; con los valores por defecto coincide, pero cambiar la configuración habría elegido el vértice de referencia equivocado sin aviso.
- El aspecto y la inclinación se miden en el plano de unidades mixtas (ρ [m], φ [rad]), de modo que los valores «aspecto ≈ 8 » no son invariantes geométricos. Es una convención válida porque se aplica idéntica a ambos métodos, pero no debe leerse como una propiedad absoluta.

Tras aplicar las correcciones, se verificó que la rejilla publicada de PR#4 quedó **bit-idéntica** (diferencia relativa máxima 0,00) en cinco poses de control, y que los PNG se regeneraron byte-idénticos.

5. Estado final y decisiones

Decisión 1 (El simulador pasa a ser la implementación vectorizada en PyTorch). El simulador de transientes se reemplaza por `simulator.py`, una implementación en PyTorch que procesa **bloques de triángulos contra todos los píxeles del SPAD a la vez** (parámetro `triangle_chunk_size`, por defecto 256), en lugar de enviar los triángulos uno por uno. Detecta CUDA y cae a CPU si no está disponible. Además de la simulación, genera conjuntos de datos completos en HDF5 (una carpeta por escena con `transient.hdf5`, `scene.obj` y `meta.json`), con muestreo aleatorio validado de posiciones de objeto e interfaz de línea de comandos.

Se adoptó **tal cual**, sin cambios de diseño; la única modificación es que `DEFAULT_OBJECT_FOLDER` apunta a la carpeta `objects/` real de este repositorio en lugar de `obs/`. La función `simulation(...)` devuelve **dos** valores, (`y_meas_vec_noisy`, `params`), tras aplicar la reorientación `np.roll(..., shift=1, axis=1)`.

Decisión 2 (La grilla de referencia baja de 64×64 a 32×32). El *baseline* pasa a `cam_pixel_dim=32`. El resto de los parámetros se mantiene coherente con lo que usaba el repositorio: FOV de cámara 0,25, $\Delta t = 3,9 \cdot 10^{-10}$ s, $I_{\text{láser}} = 1000$, paredes ocultas, sin ruido, faceta `facet.obj` con $w = 0,5$ m, tasa de fondo $b = 0,1$, y pasos de diferencias finitas $(\delta_\rho, \delta_\varphi) = (0,01 \text{ m}, 0,5^\circ)$. Reducir la grilla a la mitad en cada eje deja 1024 píxeles en lugar de 4096, y el CRB empeora en consecuencia por un factor ≈ 2 en σ (la información de Fisher es aditiva sobre píxeles, luego σ escala como la raíz del número de píxeles).

Decisión 3 (El CRB de producción es el de FD con $\psi = (\rho, \varphi)$ y Fisher 2×2 ; σ_h deja de existir). Esta es la consecuencia de modelado más importante de la decisión 1, y conviene enunciarla sin rodeos.

El simulador nuevo escala la malla del objeto con

$$\text{escala} = \min\left(\frac{w}{\text{extents}_x}, \frac{1,1}{\text{extents}_z}\right), \quad (7)$$

y **no lee en ningún momento una clave h** : la altura del objeto no es una entrada controlable, sino un resultado de la geometría de la malla y de la anchura w . Por tanto **no existe la derivada $\partial g / \partial h$ y no puede definirse σ_h** . El vector de parámetros del CRB queda reducido a $\psi = (\rho, \varphi)$, la matriz de información de Fisher es 2×2 , y hay que llamar al CRB con `estimate_height=False`.

Consecuencias concretas: desaparece toda la familia de resultados «CRB(ρ, φ, h) / Fisher 3×3 » que aparecía en PR#1 (figura 2b) y la comparación 2 frente a 3 parámetros. Si en el futuro se quiere recuperar σ_h , hay que *añadir* al simulador una entrada de altura explícita —lo que sería un cambio de diseño del código aportado, no una adaptación—; mientras eso no ocurra, la altura es un parámetro de nuisance fijado por la geometría, no estimable.

5.1. Cómo se conectan el CRB y el simulador nuevo

Hay una incompatibilidad de interfaz que merece quedar registrada. La función

```
crb_polar_functions.simulate_facet_signal_expected
```

desempaqueta **tres** valores del *callable* de simulación,

```
_, y_signal, params = simulation_fn(**kwargs),
```

herencia del simulador del *notebook*, que devolvía (ruidoso, limpio, params). El simulador nuevo devuelve **dos**.

Se adaptó el **lado del CRB**, no el simulador. Un adaptador delgado,

`crb_fd_baseline.simulation_fn_for_crb,`

llama a `simulator.simulation` y re-emite su par (y, params) como la terna (y, y, params) que el CRB espera.

La adaptación es legítima y no pierde información porque el CRB siempre invoca el *forward* con `force_no_noise=True`, que fija `add_noise=False`; en ese caso el único transiente devuelto es la señal esperada sin ruido, así que puede ocupar legítimamente ambas ranuras. El adaptador falla de forma explícita si se le pasa `add_noise=True`. Con este diseño, `crb_polar_functions.py` **no se modificó en absoluto**.

Dos notas menores de compatibilidad: el diccionario de posición de objeto que construye el CRB incluye claves `h` y `yaw` que el simulador nuevo simplemente ignora (no fallan); y el *baseline* usa `torch.float64` porque las diferencias centrales con pasos de 1 cm y $0,5^\circ$ producen diferencias de intensidad por *bin* en las que el redondeo de `float32` sería una fracción apreciable.

5.2. Resultados del CRB por FD con el *baseline* 32×32

La tabla 4 es el resultado de referencia del repositorio a partir de ahora. Se obtuvo ejecutando `compute_crb_fd_baseline.py` completo: 15 poses, 5 evaluaciones del *forward* por pose (una central y cuatro perturbadas), en 77,2s totales, es decir 5,15s por pose en CPU con `float64`. Cada evaluación produce un transiente de $32 \times 32 \times 95$ *bins*, o 97 280 observaciones de Poisson.

Tabla 4: *Baseline* final: $\text{CRB}(\rho, \varphi)$ por diferencias finitas sobre el simulador PyTorch, grilla 32×32 , Fisher 2×2 . La columna $\rho \sigma_\varphi$ es el error tangencial y $r_{\rho\varphi}$ la correlación entre ambos parámetros. No hay columna σ_h : no está definida (decisión 3).

ρ [m]	φ [°]	σ_ρ [m]	σ_φ [°]	$\rho \sigma_\varphi$ [m]	$r_{\rho\varphi}$
0.50	30	0.00136	1.003	0.008757	-0.731
0.50	60	0.001352	0.8721	0.00761	-0.583
0.50	90	0.001556	0.7535	0.006576	-0.435
0.50	120	0.002223	0.8106	0.007074	-0.402
0.50	150	0.003695	1.087	0.009487	-0.403
1.00	30	0.006852	3.717	0.06488	-0.564
1.00	60	0.007162	2.714	0.04736	-0.379
1.00	90	0.008795	2.574	0.04493	-0.283
1.00	120	0.01318	2.67	0.04661	-0.250
1.00	150	0.02271	4.159	0.07259	-0.256
1.50	30	0.02566	10.77	0.282	-0.452
1.50	60	0.02775	7.32	0.1916	-0.280
1.50	90	0.03482	7.29	0.1908	-0.209
1.50	120	0.05176	7.381	0.1932	-0.191
1.50	150	0.09031	11.69	0.3059	-0.184

Lectura de los resultados. Las tres tendencias son las esperadas y coinciden cualitativamente con todo el estudio anterior:

- **La precisión se degrada rápido con la distancia.** De $\rho = 0,5$ a $\rho = 1,5$ m, σ_ρ crece más de un factor 20 (1,4 mm $\rightarrow$ 2,6–9,0 cm). El transiente de dos rebotes decae como $1/(d_1^2 d_2^2)$, así que la señal se desvanece muy rápido.

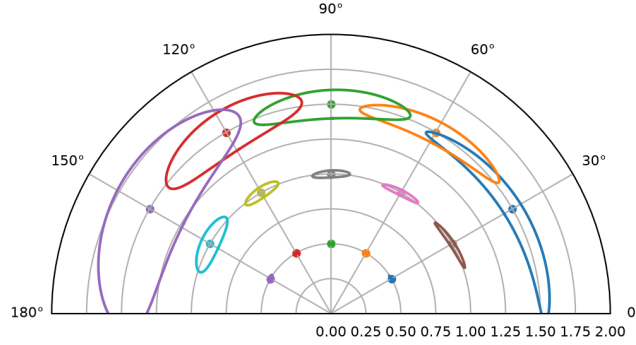


Figura 6: Regiones 3σ del $\text{CRB}(\rho, \varphi)$ por diferencias finitas sobre el simulador PyTorch con la grilla 32×32 . Es la figura de referencia del repositorio; se regenera con `python3 compute_crb_fd_baseline.py`.

- **El ángulo es más incierto en los extremos.** Para cada ρ , σ_φ es mínima cerca de $\varphi = 90^\circ$ (frente al sensor) y máxima en $\varphi = 30^\circ$ y 150° . En $\varphi = 150^\circ$ el objeto está más ocluido por la arista y aporta menos información azimutal.
- **ρ y φ están anticorrelacionados** ($r_{\rho\varphi}$ entre $-0,18$ y $-0,73$), lo que es la razón por la que las burbujas de la figura 6 aparecen inclinadas y no alineadas con los ejes: un error de rango puede compensarse parcialmente con un error de ángulo manteniendo un tiempo de vuelo similar.
- **El error tangencial $\rho\sigma_\varphi$ domina el error radial** en todas las poses, por un factor de 4 a 6. La cámara de esquina localiza el rango mucho mejor que el azimut, que es exactamente la anisotropía que las elipses muy alargadas (aspecto ≈ 8) venían mostrando en todo el estudio.

Comparando con la referencia FD anterior a 64×64 (tabla 3, columnas FD), la degradación por bajar a 32×32 es de un factor $\approx 1,9$ – $2,4$ en σ_ρ y $\approx 1,6$ – $1,9$ en σ_φ , consistente con la aditividad de la información de Fisher sobre los píxeles.

5.3. Qué quedó en el repositorio y qué se retiró

La rama consolidada conserva únicamente la documentación, el simulador nuevo, el *pipeline* del CRB por diferencias finitas y sus utilidades y objetos. Todo lo retirado se comprobó previamente sin referencias (con `grep`), y después de retirarlo se volvió a ejecutar el *pipeline* completo: el *smoke test* del simulador y un recálculo con `-force` de las 15 poses reproducen sus números anteriores, y `plots/crb_fd_baseline_32.png` se regenera byte-idéntico.

Retirado.

- `pyerti.py` — el simulador antiguo en CPU. Superado por `simulator.py`; nadie lo importaba y su carpeta de objetos apuntaba a un directorio `NLOS-Simulator/` inexistente.
- `regenerate_crb_standard_regions.py` — obtenía la función de simulación ejecutando (`exec`) las celdas 1, 3, 5, 7, 8 del *notebook*. Superado por `compute_crb_fd_baseline.py`. Calculaba además el CRB de tres parámetros, que el simulador nuevo no puede sostener (decisión 3).

- `simulation_polar_clean.ipynb`, `simulation.ipynb` y `validation.ipynb` — unos 9 MB de *notebooks*. El primero solo lo referenciaba el *script* anterior; los otros dos, nada. Su contenido queda documentado en este reporte.
- `plots/crb_standard_regions.png` y `plots/crb_standard_regions_fixed_h.png` — salidas del *script* retirado, calculadas sobre el simulador antiguo a 64×64 . Las versiones mejoradas de esos mismos resultados se conservan en `docs/figs_consolidado/` y están incrustadas en la figura 2.

Conservado a propósito, aunque el *pipeline* no lo use.

- `sr_crb.py` — un barrido de resolución de cámara, directamente pertinente a la elección de la grilla de 32×32 . Está **roto** en este momento: `utils/crb_utils.py` es un archivo de 0 bytes, de modo que `sr_crb.py` ni se importa. Se conserva porque documenta la interfaz que se pretendía (`SimulationConfig`, `run_sweep`, `plot_range_angle_sweep`) y porque el barrido sería útil para justificar cuantitativamente la grilla.
- `objects/validation.ipynb` — inerte, porque `get_obj_files` solo busca `*.obj`, y situado en una ruta protegida.
- `utils/NLOS_simulator.py` y `utils/crb_fix.py` — código de simulación anterior, bajo `utils/`, que se conserva íntegro.

6. Catálogo de figuras

Todas las figuras de este documento están en `docs/figs_consolidado/` y provienen de las ramas indicadas. Se conservan aquí porque las ramas se retiran.

Archivo	Origen	Qué muestra
<code>fd_baseline_32.png</code>	actual	Figura de referencia actual: burbujas 3σ del $\text{CRB}(\rho, \varphi)$ por FD sobre el simulador PyTorch a 32×32 (figura 6).
<code>pr1_fd_fixed_h.png</code>	PR#1	$\text{CRB}(\rho, \varphi)$ por FD, Fisher 2×2 , simulador antiguo a 64×64 .
<code>pr1_fd_with_h.png</code>	PR#1	$\text{CRB}(\rho, \varphi, h)$ por FD, Fisher 3×3 , proyectado en (ρ, φ) . Ya no reproducible: el simulador nuevo no tiene entrada de altura (decisión 3).
<code>pr1_fd_compare_2_vs_3.png</code>	PR#1	Comparación lado a lado y superpuesta de 2 frente a 3 parámetros. También histórica, por la misma razón.
<code>pr2_analytic_fixed_h.png</code>	PR#2	Burbujas del CRB analítico base, en su propia escala (no comparable en magnitud).
<code>pr2_shapenorm.png</code>	PR#2	Superposición normalizada en forma, analítico (verde) frente a FD (rojo): la única comparación honesta en esa rama.
<code>smoothing_binning.png</code>	PR#4	El reemplazo $[\cdot] \rightarrow \text{erf}$, con la conservación de masa del <i>kernel</i> .
<code>smoothing_occlusion.png</code>	PR#4	El reemplazo escalón $\rightarrow$ sigmoide, con la penumbra de un píxel ($\kappa = 256$).
<code>smoothing_foreshortening.png</code>	PR#4	El reemplazo $\text{máx}(0, \cdot) \rightarrow \text{softplus}$ en los cosenos.
<code>pr3_overlay.png</code>	PR#3	Superposición a magnitud real (a)+(b) sobre FD: escalas correctas, elipses sistemáticamente pequeñas por el sesgo de cuadratura puntual (hallazgo 1).

Archivo	Origen	Qué muestra
<code>pr4_overlay.png</code>	PR#4	Superposición a magnitud real (a)+(b)+(c) sobre FD: coincidencia en ambos ejes dentro del 10 %.
<code>pr4_overlay_rho15.png</code>	PR#4	El mismo detalle restringido a $\rho = 1,5$ m, el caso más difícil.

6.1. Documentos originales y dónde quedan sus contenidos

- `docs/sustento_teorico_crb.tex` (en main): sustento teórico del CRB de la implementación. **Se conserva en el repositorio.**
- `docs/analytic_forward_crb.tex` (PR#2, PR#3, PR#4, una versión por rama): la construcción paso a paso del *forward* diferenciable, la derivada analítica, y la justificación matemática con los teoremas de la función implícita, Rademacher, convergencia dominada, Leibniz y *mollifiers*. Sus contenidos se resumen en la sección 2 y sus resultados en la sección 3.
- `docs/guia_intuitiva.tex` (PR#4): guía en lenguaje llano, con el mapa de ramas y el catálogo de *plots*. Su material se refleja en las secciones 2 y 6.
- `docs/analytic_vs_fd_comparison.txt` (PR#2, PR#3, PR#4): las tablas numéricas comparativas. Reproducidas íntegras en las tablas 1, 2 y 3.

7. Cómo reproducir el estado final

1. Dependencias (la versión de CPU de `torch` basta: el código detecta CUDA y cae a CPU si no la hay):

```
pip install -r requirements.txt
```

2. Simulación suelta o generación de un conjunto de datos en HDF5:

```
python3 simulator.py -num-scenes 4 -cam-pixel-dim 32 -device auto
```

3. CRB de referencia:

```
python3 compute_crb_fd_baseline.py
```

Cachea en `plots/crb_fd_baseline_32.pkl` (ignorado por git) y escribe `plots/crb_fd_baseline_32.png`. Con `-force` recomputa ignorando la caché, y con `-cam-pixel-dim N` evalúa otra grilla.